

Cifragem de Hill

Mário Leite

Você sabia que uma das cifras clássicas mais fascinantes usa apenas matrizes e aritmética modular? Estou falando da **Cifra de Hill**, criada pelo matemático Lester S. Hill em 1929!

Um pouco de história - Em uma época em que a maioria das cifras substituía uma letra por vez, Hill teve uma ideia revolucionária: cifrar *blocos* de letras ao mesmo tempo. Foi a primeira cifra poligráfica prática para mais de três símbolos — um verdadeiro marco na história da criptografia.

Como funciona? O processo é pura álgebra linear: cada letra vira um número (A=0, B=1, ..., Z=25); O texto é dividido em blocos de tamanho n (aqui, pares de letras); cada bloco vira um vetor e é multiplicado pela matriz-chave; o resultado passa pelo módulo 26 e volta a ser letra. Para decifrar, basta multiplicar pela matriz *inversa* da chave (**mod 26**). Simples e elegante!

Exemplo com a chave $\begin{bmatrix} 2 & 1 \\ 5 & 3 \end{bmatrix}$ - Pegue o bloco "OP" $\rightarrow$ O=14, P=15. Multiplicando pela chave: • 1ª letra: $(14 \times 2 + 15 \times 5) \bmod 26 = 103 \bmod 26 = 25 \rightarrow$ "Z" • 2ª letra: $(14 \times 1 + 15 \times 3) \bmod 26 = 59 \bmod 26 = 7 \rightarrow$ "H" "OP" vira "ZH". Repare que as duas letras do bloco influenciam o resultado - esse é o poder da difusão!

Mas atenção ao detalhe! Nem toda matriz serve: a chave precisa ser *invertível* em **mod 26**. Ou seja, o determinante deve ser coprimo de 26 (ter inverso modular). Senão, a mensagem jamais poderá ser decifrada.

Pontos fortes - Disfarça a frequência das letras (adeus, análise de frequências!); simples de implementar em qualquer linguagem; ótima para ensinar matrizes, determinantes e aritmética modular.

Pontos fracos - Vulnerável a ataques de texto claro conhecido: com poucos pares (original, cifrado), descobre-se a chave; exige preenchimento (*padding*) quando o texto não é múltiplo do bloco — geralmente a letra "X"; não é segura para uso real hoje — seu valor é educacional.

Mão na massa! Implementei a cifra em Python usando uma matriz **2x2** como chave. O programa converte o texto, multiplica as matrizes em **mod 26**, calcula a inversa pela matriz adjunta e devolve a mensagem original intacta. Ver a teoria ganhando vida na tela é gratificante demais!

Conclusão A Cifra de Hill não protege segredos bancários, mas é uma ponte maravilhosa entre a matemática e a computação. Se você estuda álgebra linear ou criptografia, implementá-la é o exercício perfeito.

E você, já implementou alguma cifra clássica? Pelo menos a Cifragem de Cesar!?

```
'''
CifragemHill.py
-----

Faz a cifragem de um texto usando o "Cifra de Hill", usando uma matriz quadrada
2x2 invertível (tem que ser invertível) como chave da cifragem.
-----
'''

def Cifrar(texto, chave):
    # Remove espaços e caracteres não alfabéticos
    texto = ''.join(caracter if caracter.isalnum() or caracter.isspace() else ''
    for caracter in texto)

    # Converte letras em números (A=0, B=1, ..., Z=25)
    # O upper() já foi tratado no programa principal
    LstNumeros = [ord(letra) - ord('A') for letra in texto if letra.isalpha()]

    # Garante que o texto seja um múltiplo do tamanho da matriz-chave
    while(len(LstNumeros) % len(chave) != 0):
        LstNumeros.append(23) # Adiciona "X" (índice 23) como preenchimento

    # Converte a lista de números em uma matriz
    matrizTexto = Remodelar(LstNumeros, (-1, len(chave)))

    # Multiplica a matriz pela matriz-chave
    matrizCifrada = MultiplicarMatrizes(matrizTexto, chave, 26)

    # Converte a matriz cifrada de volta para letras
    textoCifrado = ''.join([chr(num + ord('A')) for linha in matrizCifrada for
        num in linha])

    return textoCifrado

#-----
def Decifrar(textoCifrado, chave):
    mod = 26
    matrizchave = chave
    matrizchaveInv = InverterMat26(matrizchave)

    LstNumeros = [ord(letra) - ord('A') for letra in textoCifrado]

    # Cria a matriz do texto cifrado
    matrizTextoCifrado = Remodelar(LstNumeros, (len(textoCifrado) // len(chave),
        len(chave)))

    # Multiplica pela matriz inversa para decifrar
    matrizTextoDecifrado = MultiplicarMatrizes(matrizTextoCifrado,
        matrizchaveInv, mod)

    textoDecifrado = ''

    # Converte os números de volta para letras
    for i in range(len(matrizTextoDecifrado)):
        textoDecifrado += ''.join(chr(num % mod + ord('A')) for num in
            matrizTextoDecifrado[i])

    # Remove o último "X" se estiver presente no final do texto decifrado (padding)
    if textoDecifrado.endswith('X'):
        textoDecifrado = textoDecifrado[:-1]

    return textoDecifrado

#-----
def Remodelar(lista, forma):
    return [lista[i:i+forma[1]] for i in range(0, len(lista), forma[1])]
```

```

#-----
# FUNÇÃO ADICIONADA PARA RESOLVER O ERRO
def MultiplicarMatrizes(matrizA, matrizB, mod):
    linhasA = len(matrizA)
    colunasA = len(matrizA[0])
    colunasB = len(matrizB[0])

    resultado = []
    # Percorre as linhas da matriz A
    for i in range(linhasA):
        linha Resultado = []
        # Percorre as colunas da matriz B
        for j in range(colunasB):
            soma = 0
            # Multiplica linha por coluna
            for k in range(colunasA):
                soma += matrizA[i][k] * matrizB[k][j]
            # Aplica o módulo 26 para manter no alfabeto
            linha_resultado.append(soma % mod)
        resultado.append(linha_resultado)
    return resultado

#-----
def InverterMat26(matriz):
    det = (matriz[0][0]*matriz[1][1] - matriz[0][1]*matriz[1][0]) % 26
    detInv = InverterMod26(det)

    if detInv is None:
        raise ValueError("Erro: A matriz-chave não é invertível (det = 0)")

    adjacente = [[matriz[1][1], -matriz[0][1]], [-matriz[1][0], matriz[0][0]]]
    inversa = [[(adjacente[i][j]*detInv) % 26 for j in range(2)] for i in range(2)]
    return inversa

#-----
def InverterMod26(numero):
    for i in range(26):
        if (numero * i) % 26 == 1:
            return i
    return None

=====
# Programa principal
if __name__ == "__main__":
    LstchaveMat = [[2, 1], [5, 3]] #a matriz TEM que ser invertível
    textOriginal = ""

    while(len(textOriginal) < 2):
        # Correção de sintaxe: adicionado o ']' que faltava no texto do input
        textOriginal = input("Digite o texto a ser cifrado [sem espaços]: ")

    textOriginal = textOriginal.upper()
    textOriginal = textOriginal.replace(" ", "")
    textoCifrado = Cifrar(textOriginal, LstchaveMat)
    textoDecifrado = Decifrar(textoCifrado, LstchaveMat)
    print('-' * 40)
    print(f'Texto Original: {textOriginal}')
    print(f'Texto Cifrado: {textoCifrado}')
    print(f'Texto Decifrado: {textoDecifrado}')
    print('-' * 40)
#Fim do programa "CifragemHill" -----

```

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> == RESTART: H:/Drive D/Cantinho da Programação/Códigos/Python/CifragemHill.py ==
Digite o texto a ser cifrado [sem espaços]: opacotefoientregue
-----
Texto Original:  OPACOTEF OIENTREGUE
Texto Cifrado:   ZHKGTTHTQMVRTSMWIG
Texto Decifrado: OPACOTEF OIENTREGUE
-----

>>> |
```

Ln: 11 Col: 0

Figura 1 - Saída do programa “CifragemHill”